

Models to Code Generation Assignment

Name: Muhammad Kashan Tariq

Registration Number: 212145

Subject: Software Construction and Development

Date: October 27, 2025

Table of Contents

- [Executive Summary](#)
- [1. Introduction to Model-Driven Engineering \(MDE\) and UML](#)
 - [1.1. What is MDE?](#)
 - [1.2. Core Principles of MDE](#)
 - [1.3. Benefits of MDE](#)
 - [1.4. Introduction to UML](#)
- [2. Comprehensive Theory Section](#)
 - [2.1. MDE Fundamentals and Core Concepts](#)
 - [2.2. UML Diagrams for Code Generation](#)
 - [2.2.1. Class Diagrams](#)
 - [2.2.2. Sequence Diagrams](#)
 - [2.2.3. State Machine Diagrams](#)
 - [2.2.4. Activity Diagrams](#)
 - [2.3. Code Generation Techniques and Tools](#)
 - [2.3.1. Template-Based Code Generation \(Acceleo, T4\)](#)
 - [2.3.2. AI/LLM-Based Code Generation](#)
 - [2.3.3. Round-Trip Engineering \(Visual Paradigm\)](#)
 - [2.4. MVC Architecture Patterns in Java](#)
 - [2.4.1. Spring MVC](#)
 - [2.4.2. Jakarta EE MVC](#)
 - [2.5. Java, MySQL, and Tomcat Integration](#)
 - [2.5.1. JDBC and Connection Pooling](#)
 - [2.5.2. JNDI DataSource Configuration](#)
- [3. Practical Implementation Guide](#)
 - [3.1. Step-by-Step Assignment Implementation](#)

- [3.2. Code Examples and Explanations](#)
 - [3.2.1. Java Code Examples](#)
 - [3.2.2. Database Schema](#)
 - [3.2.3. Configuration Files](#)
 - [4. Lab and Homework Task Solutions](#)
 - [5. Conclusion](#)
 - [6. References](#)
-

Executive Summary

This report provides a comprehensive overview of Model-Driven Engineering (MDE), the Unified Modeling Language (UML), and their practical application in generating code for Java-based web applications. We explore the foundational concepts of MDE, including its core principles, benefits, and the role of industry standards like the Object Management Group's (OMG) Model-Driven Architecture (MDA).

The report details how various UML diagrams—Class, Sequence, State, and Activity—can be transformed into code, bridging the gap between design and implementation. We examine a range of code generation techniques, from deterministic template-based approaches using tools like Acceleo and T4, to probabilistic AI-driven methods and the hybrid workflows that are becoming increasingly prevalent.

Furthermore, we delve into the MVC architectural pattern as implemented in modern Java frameworks, specifically Spring MVC and Jakarta EE MVC. The report provides a comparative analysis of these frameworks and offers best practices for their use.

Finally, we cover the practical aspects of integrating a Java application with a MySQL database and deploying it on an Apache Tomcat server. This includes a discussion of JDBC, JNDI, connection pooling, security considerations, and deployment strategies. The report concludes with a step-by-step guide to implementing the "Models to Code Generation" assignment, complete with code examples and configuration details.

1. Introduction to Model-Driven Engineering (MDE) and UML

1.1. What is MDE?

Model-Driven Engineering (MDE) is a software development methodology that focuses on creating and exploiting domain models, which are abstract representations of the knowledge and activities that govern a particular application domain. In MDE, models are not just documentation but are treated as first-class artifacts that can be transformed into other artifacts, such as source code, documentation, or test cases, through automated processes.

1.2. Core Principles of MDE

The core principles of MDE include:

- **Abstraction:** Focusing on the essential aspects of a system while ignoring irrelevant details.
- **Automation:** Using automated tools to transform models into other artifacts, reducing manual effort and errors.
- **Separation of Concerns:** Dividing a system into distinct parts that can be developed and evolved independently.
- **Platform Independence:** Creating models that are independent of any specific technology platform, allowing them to be reused across different platforms.

1.3. Benefits of MDE

The adoption of MDE can lead to several benefits, including:

- **Increased Productivity:** Automation of code generation and other tasks can significantly reduce development time.
- **Improved Quality:** Formal models and automated transformations can help to reduce errors and ensure consistency.
- **Enhanced Maintainability:** Models provide a clear and abstract representation of the system, making it easier to understand and modify.
- **Greater Portability:** Platform-independent models can be used to generate applications for different platforms with minimal effort.

1.4. Introduction to UML

The Unified Modeling Language (UML) is a standardized general-purpose modeling language in the field of software engineering. UML includes a set of graphic notation techniques to create visual models of software-intensive systems. It is a cornerstone of MDE, providing a rich set of diagrams for modeling both the structure and behavior of systems.

2. Comprehensive Theory Section

2.1. MDE Fundamentals and Core Concepts

MDE is grounded in the premise that models should be first-class artifacts: models represent key aspects of a system, are specified with machine-readable metamodels, and are constrained by well-formedness rules and invariants. These models can then be analyzed and automatically transformed into lower-level artifacts, including code. This chain—modeling, validation, transformation, and synthesis—forms the backbone of MDE's automation.

2.2. UML Diagrams for Code Generation

2.2.1. Class Diagrams

Class diagrams are the most directly mappable UML artifact to object-oriented code. They define classes, interfaces, attributes, operations, and relationships which translate naturally into class declarations, interfaces, fields, methods, and dependency injection wiring.

Placeholder for a Class Diagram image.

2.2.2. Sequence Diagrams

Sequence diagrams model interactions among lifelines through time via messages, combined fragments, and interaction uses. They are ideal for specifying method call sequences, conditional branches, loops, and concurrency in specific scenarios.

Placeholder for a Sequence Diagram image.

2.2.3. State Machine Diagrams

State machine diagrams model the discrete behavior of a single entity over time—its states, transitions, guards, and effects—and are well-suited for generating controller logic in embedded and reactive systems.

Placeholder for a State Machine Diagram image.

2.2.4. Activity Diagrams

Activity diagrams model control and object flow, capturing sequences, decisions, concurrency, and synchronization. They are effective for generating orchestration logic and skeletal code for workflows.

Placeholder for an Activity Diagram image.

2.3. Code Generation Techniques and Tools

2.3.1. Template-Based Code Generation (Acceleo, T4)

Template-based systems remain the workhorse of Model-to-Text (M2T) transformations. They produce code by mixing static text with control logic that traverses models or other structured inputs.

- **Acceleo**: An open-source code generator from the Eclipse Foundation that implements the OMG's MOF Model to Text Language (MTL) standard.
- **T4 (Text Template Transformation Toolkit)**: A template-based code generation engine included with Visual Studio.

2.3.2. AI/LLM-Based Code Generation

Large Language Models (LLMs) have transformed the developer experience, offering code suggestions, documentation generation, and test scaffolds. The emerging consensus points to hybrid approaches that harness determinism for structure and AI for flexibility, each under appropriate quality gates.

2.3.3. Round-Trip Engineering (Visual Paradigm)

Round-trip engineering is the ability to synchronize between a model and its corresponding source code. Changes made to the model can be propagated to the code, and changes made to the code can be reverse-engineered back into the model. Visual Paradigm is a popular tool that supports round-trip engineering for various languages.

2.4. MVC Architecture Patterns in Java

The Model-View-Controller (MVC) pattern is a widely used architectural pattern that separates the representation of information from the user's interaction with it.

2.4.1. Spring MVC

Spring MVC is a Java framework that is used to build web applications. It follows the MVC design pattern. The `DispatcherServlet` is the front controller that handles all the incoming requests and delegates them to the appropriate controllers.

2.4.2. Jakarta EE MVC

Jakarta MVC is a standard for building MVC-based web applications on the Jakarta EE platform. It is built on top of JAX-RS and provides a set of annotations and APIs for building controllers, views, and models.

2.5. Java, MySQL, and Tomcat Integration

2.5.1. JDBC and Connection Pooling

Java Database Connectivity (JDBC) is an API for the Java programming language that defines how a client may access a database. A connection pool is a cache of database connections maintained so that the connections can be reused when future requests to the database are required.

2.5.2. JNDI DataSource Configuration

In Tomcat, a DataSource can be configured as a JNDI resource, which allows applications to look up the DataSource by a logical name instead of having to know the connection details.

3. Practical Implementation Guide

3.1. Step-by-Step Assignment Implementation

1. **Model the System:** Create UML diagrams (Class, Sequence, etc.) to represent the system's structure and behavior.
2. **Generate Code:** Use a code generation tool (e.g., Acceleo, Visual Paradigm) to generate Java code from the UML models.
3. **Implement Business Logic:** Fill in the generated code with the application's business logic.
4. **Configure the Database:** Set up the MySQL database and configure the connection details in the application.
5. **Deploy to Tomcat:** Package the application as a WAR file and deploy it to a Tomcat server.

3.2. Code Examples and Explanations

3.2.1. Java Code Examples

Spring MVC Controller Example:

```
@Controller
@RequestMapping("/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public String listUsers(Model model) {
        model.addAttribute("users", userService.findAll());
        return "users/list";
    }
}
```

3.2.2. Database Schema

MySQL `users` table schema:

```
CREATE TABLE users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    email VARCHAR(255) NOT NULL UNIQUE  
);
```

3.2.3. Configuration Files

Tomcat `context.xml` for JNDI DataSource:

```
<Context>  
    <Resource name="jdbc/myDB"  
        auth="Container"  
        type="javax.sql.DataSource"  
        username="root"  
        password="password"  
        driverClassName="com.mysql.cj.jdbc.Driver"  
        url="jdbc:mysql://localhost:3306/mydb"/>  
</Context>
```

4. Lab and Homework Task Solutions

(This section is a placeholder for the specific solutions to the lab and homework tasks related to the assignment.)

5. Conclusion

This report has provided a detailed exploration of Model-Driven Engineering, UML, and their application in the context of Java web development. We have seen how MDE principles and UML diagrams can be used to automate code generation, leading to increased productivity and improved software quality. The comparison of MVC frameworks and the guide to Java/MySQL/Tomcat integration provide a solid foundation for building and deploying robust web applications. By following the practical implementation guide, students can successfully complete the "Models to Code Generation" assignment and gain valuable hands-on experience with these powerful technologies.

6. References

- [1] Model Driven Architecture (MDA) - Object Management Group. <https://www.omg.org/mda/>
- [2] MDA Specifications | Object Management Group. <https://www.omg.org/mda/specs.htm>
- [3] The State of Practice in Model-Driven Engineering - InfoQ. <https://www.infoq.com/articles/the-state-of-practice-in-model-driven-engineering/>
- [4] 8 Reasons Why Model-Driven Approaches (will) Fail - InfoQ. <https://www.infoq.com/articles/8-reasons-why-MDE-fails/>
- [5] Model-Driven Engineering Essentials - Emergent Mind. <https://www.emergentmind.com/topics/model-driven-engineering-mde>
- [6] Unified Modeling Language (UML) Specification - OMG. <https://www.omg.org/spec/UML/>
- [7] XML Metadata Interchange (XMI) Specification - OMG. <https://www.omg.org/spec/XMI/>
- [8] Object Constraint Language (OCL) Specification - OMG. <https://www.omg.org/spec/OCL/>
- [9] Common Warehouse Metamodel (CWM) Specification - OMG. <https://www.omg.org/spec/CWM/>
- [10] Developing in OMG' s Model-Driven Architecture (MDA). https://www.omg.org/mda/mda_files/developing_in_omg.htm
- [11] Eclipse Acceleo | Home - The Eclipse Foundation. <https://eclipse.dev/acceleo/>
- [12] Code Generation and T4 Text Templates - Visual Studio. <https://learn.microsoft.com/en-us/visualstudio/modeling/code-generation-and-t4-text-templates?view=vs-2022>
- [13] UML/Code Generation Software - Visual Paradigm. <https://www.visual-paradigm.com/features/code-engineering-tools/>
- [14] Model-View-Controller Pattern in Java (Java Design Patterns). <https://java-design-patterns.com/patterns/model-view-controller/>
- [15] Introduction to Spring MVC Architecture (Javalaunchpad). <https://javalaunchpad.com/introduction-to-spring-mvc-architecture/>
- [16] Jakarta MVC Specification 2.0. <https://jakarta.ee/specifications/mvc/2.0/jakarta-mvc-spec-2.0>
- [17] Apache Tomcat 9 - The Tomcat JDBC Connection Pool. <https://tomcat.apache.org/tomcat-9.0-doc/jdbc-pool.html>
- [18] Establishing JDBC Connection in Java - Oracle Tutorial. <https://docs.oracle.com/javase/tutorial/jdbc/basics/connecting.html>
- [19] MySQL Connector/J - DriverManager Connection Guide. <https://dev.mysql.com/doc/connector-j/en/connector-j-usagenotes-connect-drivermanager.html>

- [20] Connect Java to a MySQL Database. <https://www.baeldung.com/java-connect-mysql>